

50 database-related questions and answers covering topics like primary keys, indexes, constraints, normalization, SQL queries, and optimization, with examples included.

1. **Q:** What is a primary key?

A: A column or combination of columns that uniquely identifies each row in a table. It does not allow NULL values.

Example:

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Age INT  
);
```

2. **Q:** Can a table have multiple primary keys?

A: No, a table can only have one primary key, but it can consist of multiple columns (composite key).

3. **Q:** What is a composite primary key?

A: A primary key made up of two or more columns.

Example:

```
CREATE TABLE CourseEnrollment (  
    StudentID INT,  
    CourseID INT,  
    PRIMARY KEY (StudentID, CourseID)  
);
```

4. **Q:** What is an index?

A: A database object that improves the speed of data retrieval operations at the cost of additional storage.

Example:

```
CREATE INDEX idx_name ON Students(Name);
```

5. **Q:** What is the difference between clustered and non-clustered indexes?

A:

- **Clustered Index:** Sorts and stores rows physically in the table based on the index key.
- **Non-Clustered Index:** Stores a separate structure with pointers to the data rows.

6. **Q:** How can you drop an index?

A: DROP INDEX idx_name;

7. **Q:** What are constraints in SQL?
A: Rules applied to table columns to enforce data integrity (e.g., PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL).
8. **Q:** What is the difference between UNIQUE and PRIMARY KEY constraints?
A:
- **UNIQUE:** Allows only unique values; permits one NULL value.
 - **PRIMARY KEY:** Ensures uniqueness and disallows NULL values.
9. **Q:** How do you add a FOREIGN KEY constraint?
A:
- ```
CREATE TABLE Orders (
 OrderID INT PRIMARY KEY,
 CustomerID INT,
 FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID));
```
10. **Q:** What is a DEFAULT constraint?  
**A:** Sets a default value for a column when no value is provided during insert.  
**Example:**
- ```
CREATE TABLE Employees (  
    EmpID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Salary DECIMAL(10,2) DEFAULT 5000.00 );
```
11. **Q:** What is normalization?
A: Organizing data to minimize redundancy and improve integrity.
12. **Q:** What are the normal forms?
A:
- 1NF: Eliminate duplicate columns.
 - 2NF: Eliminate partial dependencies.
 - 3NF: Eliminate transitive dependencies.
13. **Q:** What is denormalization?
A: Combining tables to improve read performance, often at the cost of redundancy.
14. **Q:** How do you retrieve all records from a table?
A: SELECT * FROM Employees;
15. **Q:** How do you filter rows based on a condition?
A: SELECT * FROM Students WHERE Age > 18;
16. **Q:** How do you sort data in ascending or descending order?
A: SELECT * FROM Employees ORDER BY Salary DESC;

17. **Q:** What is the difference between WHERE and HAVING?

A:

- **WHERE:** Filters rows before grouping.
- **HAVING:** Filters groups after aggregation.

18. **Q:** What is an INNER JOIN?

A: Returns rows with matching values in both tables.

Example:

```
SELECT Orders.OrderID, Customers.Name
```

```
FROM Orders
```

```
INNER JOIN Customers ON Orders.CustomerID = Customers.CustomerID;
```

19. **Q:** What is a LEFT JOIN?

A: Returns all rows from the left table and matching rows from the right.

Example:

```
SELECT Students.Name, Course.CourseName
```

```
FROM Students
```

```
LEFT JOIN Course ON Students.CourseID = Course.CourseID;
```

20. **Q:** What is a transaction in SQL?

A: A sequence of operations performed as a single logical unit of work.

21. **Q:** How do you start, commit, and rollback a transaction?

A: BEGIN TRANSACTION;

```
UPDATE Accounts SET Balance = Balance - 100 WHERE AccountID = 1;
```

```
UPDATE Accounts SET Balance = Balance + 100 WHERE AccountID = 2;
```

```
COMMIT; -- or ROLLBACK;
```

22. **Q:** What are aggregate functions?

A: Functions that perform calculations on a set of values (e.g., COUNT, SUM, AVG).

Example:

```
SELECT AVG(Salary) FROM Employees;
```

23. **Q:** What is the difference between scalar and table-valued functions?

A:

- **Scalar Function:** Returns a single value.
- **Table-Valued Function:** Returns a table.

24. **Q:** What is a stored procedure?

A: A set of SQL statements stored in the database.

Example:

```
CREATE PROCEDURE GetEmployees
```

```
AS
```

```
BEGIN
```

```
    SELECT * FROM Employees;
```

```
END;
```

25. **Q:** How do you execute a stored procedure?

A:

```
EXEC GetEmployees;
```

26. **Q:** What is a trigger?

A: A set of SQL statements automatically executed in response to a specific event on a table.

Example:

```
CREATE TRIGGER trg_AfterInsert
```

```
ON Employees
```

```
AFTER INSERT
```

```
AS
```

```
BEGIN
```

```
    PRINT 'A new record was added';
```

```
END;
```

27. **Q:** What is a view?

A: A virtual table based on a query.

Example:

```
CREATE VIEW EmployeeView AS
```

```
SELECT Name, Salary FROM Employees;
```

28. **Q:** How do you update a view?

A: CREATE OR REPLACE VIEW EmployeeView AS

```
SELECT Name, Department FROM Employees;
```

29. **Q:** How do you optimize slow queries?

A:

- Use indexes.
- Avoid SELECT *.
- Use EXPLAIN to analyze queries.

30. **Q:** What is query execution plan?

A: A visual representation of how a query will execute.

Example:

```
EXPLAIN SELECT * FROM Employees;
```

31. **Q:** What is a cursor?

A: A database object used to retrieve one row at a time.

Example:

```
DECLARE cursor_name CURSOR FOR SELECT Name FROM Students;
```

32. **Q:** What are ACID properties?

A:

- **Atomicity:** Transactions are all or nothing.
- **Consistency:** Maintains database integrity.
- **Isolation:** Transactions do not interfere.
- **Durability:** Changes are permanent.

33. **Q:** What is the difference between DELETE and TRUNCATE?

A:

- **DELETE:** Removes specific rows; can be rolled back.
- **TRUNCATE:** Removes all rows; cannot be rolled back.

34. **Q:** What is a window function?

A: A function that performs a calculation across a set of table rows related to the current row without collapsing the result into a single value like aggregate functions do.

35. **Q:** What are the components of a window function?

A:

- **Function:** The calculation to perform (e.g., ROW_NUMBER(), SUM()).
- **OVER() clause:** Specifies how rows are partitioned or ordered.

36. **Q:** What does the ROW_NUMBER() function do?

A: Assigns a unique sequential integer to rows within a partition, based on the order specified.

Example:

```
SELECT Name, Department,
```

```
       ROW_NUMBER() OVER (PARTITION BY Department ORDER BY Name) AS RowNum
```

```
FROM Employees;
```

37. **Q:** What does the RANK() function do?

A: Assigns a rank to each row within a partition. Ties receive the same rank, and the next rank is skipped.

Example:

```
SELECT Name, Salary,  
       RANK() OVER (ORDER BY Salary DESC) AS Rank  
FROM Employees;
```

38. **Q:** How is DENSE_RANK() different from RANK()?

A: DENSE_RANK() assigns consecutive ranks without skipping any.

Example:

```
SELECT Name, Salary,  
       DENSE_RANK() OVER (ORDER BY Salary DESC) AS DenseRank  
FROM Employees;
```

39. **Q:** What does the NTILE() function do?

A: Divides rows into a specified number of groups and assigns a group number to each row.

Example:

```
SELECT Name, Salary,  
       NTILE(4) OVER (ORDER BY Salary DESC) AS Quartile  
FROM Employees;
```

40. **Q:** How is SUM() used as a window function?

A: Calculates the running total or sum of a partition.

Example:

```
SELECT Name, Department, Salary,  
       SUM(Salary) OVER (PARTITION BY Department ORDER BY Name) AS RunningTotal  
FROM Employees;
```

41. **Q:** What does AVG() with a window function do?

A: Computes the average within a partition.

Example:

```
SELECT Name, Department, Salary,  
       AVG(Salary) OVER (PARTITION BY Department) AS AvgSalary  
FROM Employees;
```

42. **Q:** What is LAG() in window functions?

A: Retrieves the value of a column from the previous row within the same partition.

Example:

```
SELECT Name, Salary,  
       LAG(Salary) OVER (ORDER BY Salary DESC) AS PreviousSalary  
FROM Employees;
```

43. **Q:** What is LEAD() in window functions?

A: Retrieves the value of a column from the next row within the same partition.

Example:

```
SELECT Name, Salary,  
  
       LEAD(Salary) OVER (ORDER BY Salary DESC) AS NextSalary  
  
FROM Employees;
```

44. **Q:** What is a frame specification in a window function?

A: Defines the subset of rows within a partition that the window function applies to.

45. **Q:** What is the default frame for a window function?

A: RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW.

46. **Q:** How do you define a custom frame?

A:

```
SELECT Name, Salary,  
  
       SUM(Salary) OVER (ORDER BY Salary ROWS BETWEEN 1 PRECEDING AND 1  
                        FOLLOWING) AS MovingSum  
  
FROM Employees;
```

47. **Q:** What is the difference between ROWS and RANGE in frame specifications?

A:

- **ROWS:** Works with physical row positions.
- **RANGE:** Works with logical values of the ordering column.

48. **Q:** How do you calculate a cumulative total using window functions?

A:

```
SELECT Name, Salary,  
  
       SUM(Salary) OVER (ORDER BY Salary ROWS BETWEEN UNBOUNDED PRECEDING  
                        AND CURRENT ROW) AS CumulativeTotal  
  
FROM Employees;
```

49. **Q:** Can you use multiple window functions in a single query?

A: Yes, you can use multiple window functions with different OVER() clauses.

Example:

```
SELECT Name, Salary,  
  
       RANK() OVER (ORDER BY Salary DESC) AS Rank,  
  
       SUM(Salary) OVER (PARTITION BY Department) AS TotalDeptSalary  
  
FROM Employees;
```

50. **Q:** What are practical use cases for window functions?

A:

- **Rankings:** Assigning ranks to employees based on performance.
- **Running Totals:** Calculating cumulative sales.
- **Comparisons:** Comparing current row with previous or next row (e.g., sales trends).